

Setting Up the Environment and Preprocessing the Data

```
In [1]: # Import required libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
```

```
In [2]: # Load the Titanic dataset from seaborn's built-in collection
titanic_df = sns.load_dataset("titanic")

# View the first few records
titanic_df.head()
```

```
Out[2]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton

```
In [3]: # Basic structure of the dataset
titanic_df.shape
```

```
Out[3]: (891, 15)
```

```
In [4]: # Column-wise data types and non-null counts
titanic_df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column      Non-Null Count  Dtype
---  -
0   survived    891 non-null    int64
1   pclass      891 non-null    int64
2   sex         891 non-null    str
3   age         714 non-null    float64
4   sibsp       891 non-null    int64
5   parch       891 non-null    int64
6   fare        891 non-null    float64
7   embarked    889 non-null    str
8   class       891 non-null    category
9   who         891 non-null    str
10  adult_male  891 non-null    bool
11  deck        203 non-null    category
12  embark_town 889 non-null    str
13  alive       891 non-null    str
14  alone       891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), str(5)
memory usage: 80.7 KB
```

```
In [5]: # Summary statistics of numerical columns
titanic_df.describe()
```

```
Out[5]:
```

	survived	pclass	age	sibsp	parch	fare
count	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

```
In [6]: # Count missing values in every column
titanic_df.isnull().sum()
```

```
Out[6]: survived      0
pclass      0
sex         0
age        177
sibsp      0
parch      0
fare       0
embarked    2
class      0
who        0
adult_male  0
deck      688
embark_town 2
alive      0
alone      0
dtype: int64
```

```
In [7]: # Handle missing values
```

```
# Fill missing 'age' entries with the column mean
titanic_df["age"] = titanic_df["age"].fillna(titanic_df["age"].mean())

# Fill missing 'embarked' entries with the most frequent port
titanic_df["embarked"] = titanic_df["embarked"].fillna(titanic_df["embarked"].mode()[0])

# Drop 'deck' since the majority of its values are missing
titanic_df.drop(columns=["deck"], inplace=True)

# Confirm no missing values remain (except columns we intentionally keep)
titanic_df.isnull().sum()
```

```
Out[7]: survived      0
pclass      0
sex         0
age         0
sibsp      0
parch      0
fare       0
embarked    0
class      0
who        0
adult_male  0
embark_town 2
alive      0
alone      0
dtype: int64
```

```
In [8]: # Drop redundant/duplicate descriptive columns before encoding
titanic_df.drop(columns=["class", "who", "adult_male", "alive", "alone"], inplace=True)

titanic_df.head()
```

```
Out[8]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	embark_town
0	0	3	male	22.0	1	0	7.2500	S	Southampton
1	1	1	female	38.0	1	0	71.2833	C	Cherbourg
2	1	3	female	26.0	0	0	7.9250	S	Southampton
3	1	1	female	35.0	1	0	53.1000	S	Southampton
4	0	3	male	35.0	0	0	8.0500	S	Southampton

```
In [9]: # Encode categorical text columns into numeric form
encoder = LabelEncoder()

titanic_df["sex"] = encoder.fit_transform(titanic_df["sex"])
titanic_df["embarked"] = encoder.fit_transform(titanic_df["embarked"])
titanic_df["embark_town"] = encoder.fit_transform(titanic_df["embark_town"])

titanic_df.head()
```

```
Out[9]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	embark_town
0	0	3	1	22.0	1	0	7.2500	2	2
1	1	1	0	38.0	1	0	71.2833	0	0
2	1	3	0	26.0	0	0	7.9250	2	2
3	1	1	0	35.0	1	0	53.1000	2	2
4	0	3	1	35.0	0	0	8.0500	2	2

```
In [10]: # Scale numerical features so they share a common range
scaler = StandardScaler()
cols_to_scale = ["age", "fare"]

titanic_df[cols_to_scale] = scaler.fit_transform(titanic_df[cols_to_scale])

titanic_df.head()
```

```
Out[10]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	embark_town
0	0	3	1	-0.592481	1	0	-0.502445	2	2
1	1	1	0	0.638789	1	0	0.786845	0	0
2	1	3	0	-0.284663	0	0	-0.488854	2	2
3	1	1	0	0.407926	1	0	0.420730	2	2
4	0	3	1	0.407926	0	0	-0.486337	2	2

```
In [11]: # Separate features (X) from the target label (y)
X = titanic_df.drop("survived", axis=1)
y = titanic_df["survived"]

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=7
)

print("Training set shape:", X_train.shape)
print("Testing set shape:", X_test.shape)
```

Training set shape: (668, 8)
Testing set shape: (223, 8)

```
In [12]: # Preview a sample of the training data
X_train.head()
```

```
Out[12]:
```

	pclass	sex	age	sibsp	parch	fare	embarked	embark_town
709	3	1	0.000000	1	1	-0.341452	0	0
558	1	0	0.715743	1	1	0.955306	2	2
327	2	0	0.484880	0	0	-0.386671	2	2
256	1	0	0.000000	0	0	0.946246	0	0
51	3	1	-0.669435	0	0	-0.491371	2	2

```
In [13]: # Save the cleaned and preprocessed dataset to disk
titanic_df.to_csv("titanic_preprocessed.csv", index=False)

print("Preprocessing complete. File saved as titanic_preprocessed.csv")
```

Preprocessing complete. File saved as titanic_preprocessed.csv